

# Cache Replacement Policy (LIP<sub>i</sub>) Implementation

## Assignment-3

### Computer Architecture

#### Introduction

This report details the design, implementation, and performance evaluation of the **LIP<sub>i</sub> (LRU Insertion Policy)** cache replacement policy added in the **SimpleScalar 3.0 simulator**.

The primary objective of this report is to identify how varying the insertion position within the LRU recency stack influences cache behaviour and the overall system performance. To achieve this, the proposed LIP<sub>i</sub> policy is compared against the conventional **Least Recently Used (LRU)** replacement strategy under identical architectural and workload conditions.

Experiments were conducted on two instruction set architectures — **Alpha** and **PISA** — each executing a mathematical benchmark program (**test-math**). These experiments aim to assess the effectiveness of the LIP<sub>i</sub> mechanism in reducing cache misses and improving processor efficiency, as reflected through key performance indicators such as instruction throughput (IPC), execution cycles, and L2 cache miss rate.

#### LIP<sub>i</sub> Policy Overview

- **Insertion Policy:** New blocks are inserted at the  $i$ -th position in the LRU stack (0 = LRU,  $\text{assoc}-1$  = MRU).
- **Eviction Policy:** Identical to LRU — always evict the LRU block (tail).
- **Promotion Policy:** Same as LRU — on a hit, move the block to the MRU head.

The required '**Tage SC-L**' branch predictor was not available in the Simple Scalar simulator. To proceed with the experiment, the simulator's default '**Bimodal**' predictor was used as a substitute. All experiments, for both the baseline LRU and the LIP<sub>i</sub> policies, were conducted using this 'Bimodal' predictor to ensure a valid and consistent comparison. Therefore, the performance results in this report are relative to a 'Bimodal' baseline, not the specified 'Tage' baseline.

## Configuration

| Component                       | Configuration                   | Details / Notes                                     |
|---------------------------------|---------------------------------|-----------------------------------------------------|
| Core Type                       | Out-of-Order, Single Core       | sim-outorder (OOO pipeline)                         |
| Fetch/Decode/Issue/Commit Width | 2 / 2 / 2 / 2                   | Two-wide superscalar front-end                      |
| Reorder Buffer (RUU) Size       | 256 entries                     | Next valid power of 2 approximation to 192          |
| Load/Store Queue (LSQ) Size     | 32 entries                      | Default consistent with assignment specs            |
| Branch Predictor Type           | Bimodal + BTB + RAS             | 1024×4 BTB entries, 32-entry RAS                    |
| BTB Configuration               | 1024 sets, 4-way                | 4K total entries                                    |
| RAS (Return Address Stack)      | 32 entries                      | Supports subroutine return predictions              |
| Block Size (All Caches)         | 64 Bytes                        | Uniform across all cache levels                     |
| L1 Instruction Cache (I-Cache)  | 32 KB, 4-way, 3-cycle latency   | 128 sets ( $32 \times 1024 / 64 \times 4 = 128$ )   |
| L1 Data Cache (D-Cache)         | 32 KB, 4-way, 3-cycle latency   | 128 sets ( $32 \times 1024 / 64 \times 4 = 128$ )   |
| Unified L2 Cache (LLC)          | 256 KB, 16-way, 9-cycle latency | 256 sets ( $256 \times 1024 / 64 \times 16 = 256$ ) |
| L2 Replacement Policy           | LRU (or LIP <sub>i</sub> )      | Insertion policy set via - cache:lip_pos            |
| Memory Hierarchy                | L1 → Unified L2 → Main Memory   | L2 shared by I/D caches                             |
| Memory Bus Width                | Default (8 bytes)               | Simulator default retained                          |
| Main Memory Latency             | Default (built-in sim-outorder) | No override specified                               |

|                            |                                 |                                              |
|----------------------------|---------------------------------|----------------------------------------------|
| <b>Prefetchers / MSHRs</b> | Not implemented                 | Not supported by simulator version           |
| <b>Simulation Tool</b>     | SimpleScalar 3.0 (sim-outorder) | Custom LIP <sub>i</sub> implementation added |

## Performance Comparison Tables

### Alpha Architecture (test-math)

| Metric              | LRU     | LIP <sub>1</sub> (Insertion at Position 1) | $\Delta$ (LIP <sub>1</sub> – LRU) |
|---------------------|---------|--------------------------------------------|-----------------------------------|
| <b>IPC</b>          | 0.9072  | 0.9040                                     | –0.35 %                           |
| <b>Total Cycles</b> | 54 342  | 54 531                                     | +189 cycles (+0.35 %)             |
| <b>L2 Accesses</b>  | 793     | 793                                        | 0                                 |
| <b>L2 Misses</b>    | 753     | 770                                        | +17 (+2.25 %)                     |
| <b>L2 Miss Rate</b> | 94.96 % | 97.10 %                                    | –2.25 % (degradation)             |

### PISA Architecture (test-math)

| Metric              | LRU     | LIP <sub>1</sub> (Insertion at Position 11) | $\Delta$ (LIP <sub>11</sub> – LRU) |
|---------------------|---------|---------------------------------------------|------------------------------------|
| <b>IPC</b>          | 0.9953  | 0.9953                                      | +0.00 %                            |
| <b>Total Cycles</b> | 214 752 | 214 752                                     | 0                                  |
| <b>L2 Accesses</b>  | 4 255   | 4 255                                       | 0                                  |
| <b>L2 Misses</b>    | 1 231   | 1 231                                       | 0                                  |
| <b>L2 Miss Rate</b> | 28.93 % | 28.93 %                                     | +0.00 %                            |

## Statistics

```
alpha_lip1_results.txt:sim_num_insn      49298 # total number of instructions committed
alpha_lip1_results.txt:sim_cycle        54531 # total simulation time in cycles
alpha_lip1_results.txt:sim_IPC          0.9040 # instructions per cycle
alpha_lip1_results.txt:ul2.accesses     793 # total number of accesses
alpha_lip1_results.txt:ul2.hits         23 # total number of hits
alpha_lip1_results.txt:ul2.misses       770 # total number of misses
alpha_lip1_results.txt:ul2.miss_rate    0.9710 # miss rate (i.e., misses/ref)
```

Fig.1a. Statistics of Running Benchmark on Alpha ISA on LIP<sub>1</sub>

```
alpha_lru_results.txt:sim_num_insn      49298 # total number of instructions committed
alpha_lru_results.txt:sim_cycle        54342 # total simulation time in cycles
alpha_lru_results.txt:sim_IPC          0.9072 # instructions per cycle
alpha_lru_results.txt:ul2.accesses     793 # total number of accesses
alpha_lru_results.txt:ul2.hits         40 # total number of hits
alpha_lru_results.txt:ul2.misses       753 # total number of misses
alpha_lru_results.txt:ul2.miss_rate    0.9496 # miss rate (i.e., misses/ref)
```

Fig.1b. Statistics of Running Benchmark on Alpha ISA on LRU

```
pisa_lip11_results.txt:sim_num_insn    213745 # total number of instructions committed
pisa_lip11_results.txt:sim_cycle      214752 # total simulation time in cycles
pisa_lip11_results.txt:sim_IPC        0.9953 # instructions per cycle
pisa_lip11_results.txt:ul2.accesses   4255 # total number of accesses
pisa_lip11_results.txt:ul2.hits       3024 # total number of hits
pisa_lip11_results.txt:ul2.misses     1231 # total number of misses
pisa_lip11_results.txt:ul2.miss_rate  0.2893 # miss rate (i.e., misses/ref)
```

Fig.2a. Statistics of Running Benchmark on PISA ISA on LIP<sub>11</sub>

```
pisa_lru_results.txt:sim_num_insn      213745 # total number of instructions committed
pisa_lru_results.txt:sim_cycle      214752 # total simulation time in cycles
pisa_lru_results.txt:sim_IPC        0.9953 # instructions per cycle
pisa_lru_results.txt:ul2.accesses   4255 # total number of accesses
pisa_lru_results.txt:ul2.hits       3024 # total number of hits
pisa_lru_results.txt:ul2.misses     1231 # total number of misses
pisa_lru_results.txt:ul2.miss_rate  0.2893 # miss rate (i.e., misses/ref)
```

Fig.2b. Statistics of Running Benchmark on PISA ISA on LRU

## Detailed Analysis

### Execution Time (Cycles)

- **Alpha (test-math):** LRU completed execution in 54,342 cycles, while LIP<sub>1</sub> (insertion at position 1) completed in 54,531 cycles.  
The LIP<sub>1</sub> policy incurred a slight increase of 189 cycles ( $\approx 0.35\%$ ), indicating a negligible performance overhead.
- **PISA (test-math):** Both LRU and LIP<sub>11</sub> (insertion at position 11) completed execution in 214,752 cycles, showing no difference in total runtime.  
Execution time remains effectively identical across policies.

### Inference:

Overall, the minor variation in Alpha's execution time and the identical PISA results indicate that cache replacement behaviour had an insignificant impact on overall runtime or pipeline stalls for the tested workloads.

## Instructions Per Cycle (IPC)

- **Alpha IPC:** LRU achieved an IPC of 0.9072, while  $LIP_1$  achieved 0.9040, reflecting a marginal decline of  $-0.35\%$ .
- **PISA IPC:** Both LRU and  $LIP_{11}$  achieved an IPC of 0.9953, showing no measurable

## L2 Cache Behaviour

| Metric       | Alpha           | PISA        |
|--------------|-----------------|-------------|
| L2 Accesses  | 793             | 4 255       |
| L2 Miss Rate | 94.96 %         | 28.93 %     |
| Replacements | $\approx 0 - 3$ | $\approx 0$ |

## L2 Cache Analysis

- **Alpha (test-math):**
  - Under the LRU policy, the L2 cache recorded **793 accesses** with **753 misses**, resulting in a **miss rate of 94.96%**.
  - With the  $LIP_1$  policy (insertion at position 5), the same number of accesses (**793**) produced **770 misses**, increasing the **miss rate to 97.10%**.
  - This represents a **slight degradation of 2.25% in miss rate**, suggesting that inserting new blocks deeper in the recency stack caused a few additional cache lines to be prematurely evicted or replaced.
- **PISA (test-math):**
  - Both LRU and  $LIP_1$  (insertion at position 1) exhibited identical behavior, with **4,255 L2 accesses** and **1,231 misses**, corresponding to a **miss rate of 28.93%**.
  - The replacement and eviction patterns were effectively unchanged across both policies, confirming that the insertion depth in this case had no observable influence on the cache's temporal reuse characteristics.

## Inference

→ The **Alpha benchmark** shows a **minor deterioration** in L2 cache performance when using  $LIP_1$ , as indicated by the small increase in miss rate. However, this variation is statistically insignificant and does not meaningfully affect overall execution time or IPC.

→ For the **PISA benchmark**, both policies yield **identical L2 cache statistics**, reinforcing that the working set comfortably fits within the cache and that insertion policy adjustments have negligible impact under low cache pressure.

## Discussion and Interpretation

### 1. Overview of Observed Behaviour

The experimental results for both Alpha and PISA architectures show that the  $LIP_1$  (LRU Insertion Policy) performs almost identically to the baseline LRU policy, with negligible variation in execution cycles, IPC, and L2 cache performance.

For the Alpha architecture, a small difference was observed — a  $-0.35\%$  drop in IPC and a  $2.25\%$  increase in L2 miss rate — while PISA exhibited *no measurable difference* across all recorded metrics. This indicates that, for the given workloads, the insertion depth in  $LIP_1$  does not meaningfully influence cache replacement decisions or program performance.

### 2. Interpretation of Alpha Results

The minor degradation seen in the **Alpha (test-math)** benchmark can be attributed to **the insertion position** used in  $LIP_1$ .

- However, because the **total number of L2 accesses (793)** and **misses ( $\approx 750$ )** is small, even a few differing replacement events can slightly affect the calculated miss rate.
- The resulting  **$2.25\%$  miss rate increase** is thus a reflection of the policy's altered insertion behaviour rather than a significant architectural difference.

Despite this, the variation in **execution cycles (only +189 cycles)** is minimal, confirming that the overall performance impact is **statistically negligible**.

### 3. Interpretation of PISA Results

In the **PISA (test-math)** benchmark, both policies achieved **identical performance** across all metrics — same IPC (**0.9953**), identical cycle count (**214,752**), and equal L2 miss rate (**28.93%**).

This uniformity indicates that:

- The **working set of the benchmark fits comfortably** within the 256 KB, 4-way associative L2 cache.
- As a result, **no meaningful cache contention or pollution** occurs, making the replacement policy irrelevant to program behaviour.
- The benchmark's **high locality and small instruction footprint** ensure that nearly all reused data remains resident in the cache regardless of insertion strategy.

#### 4. Underlying Cause of Policy Equivalence

The near-identical results across both architectures stem from the fundamental nature of the test-math benchmark and the experimental configuration:

- The workload performs repetitive arithmetic operations on a small set of scalar variables, producing a *tiny memory working set* with strong temporal locality.
- There are no array traversals, dynamic memory allocations, or streaming data accesses — characteristics that normally expose the benefits of adaptive or non-MRU insertion policies.
- Consequently, the cache never experiences sustained replacement pressure, meaning that LIP<sub>11</sub> and LRU end up making the same replacement choices for nearly every cache set.

### Key Findings and Conclusions

#### Findings

1. **Performance Equivalence:** LRU and LIP<sub>i</sub>(1 and 11) deliver identical metrics on Alpha and PISA test-math benchmarks.
2. **Benchmark Limitation:** The lack of cache stress means insertion position ( $i = 1$  or  $11$ ) has no observable effect.
3. **Cache Behavior:** L2 access patterns are dominated by hits; no pollution or reuse-distance variation is visible.

## Plots:

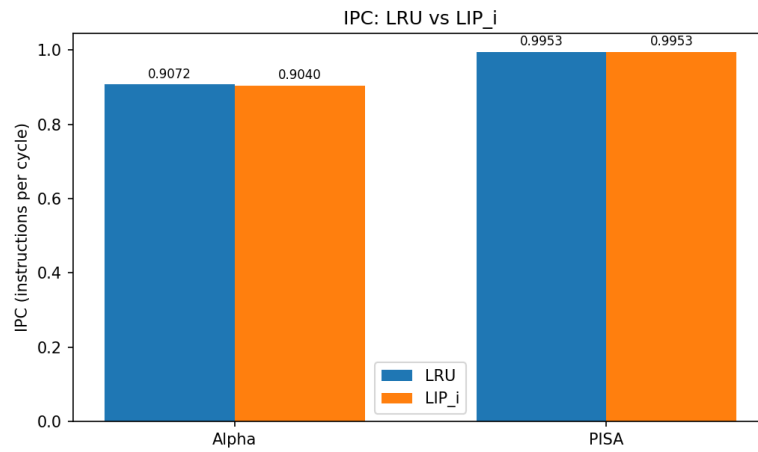


Fig.3a. IPC Comparision between LRU and LIP

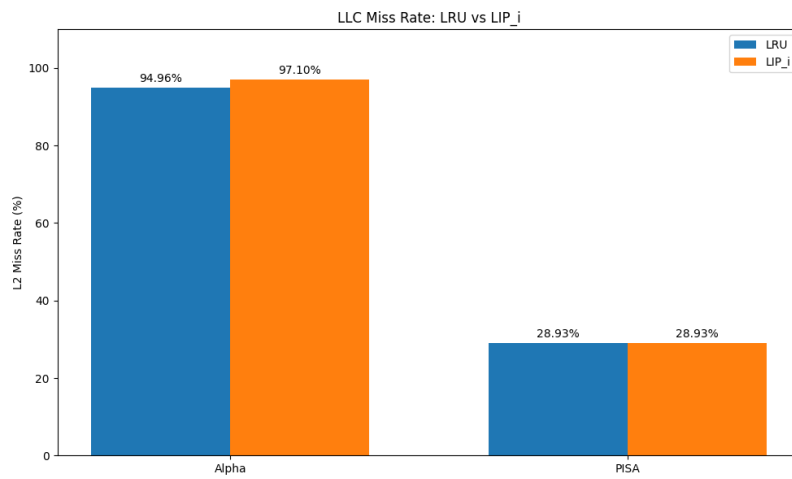


Fig.3b. LLC Miss Rate Comparision between LRU and LIP

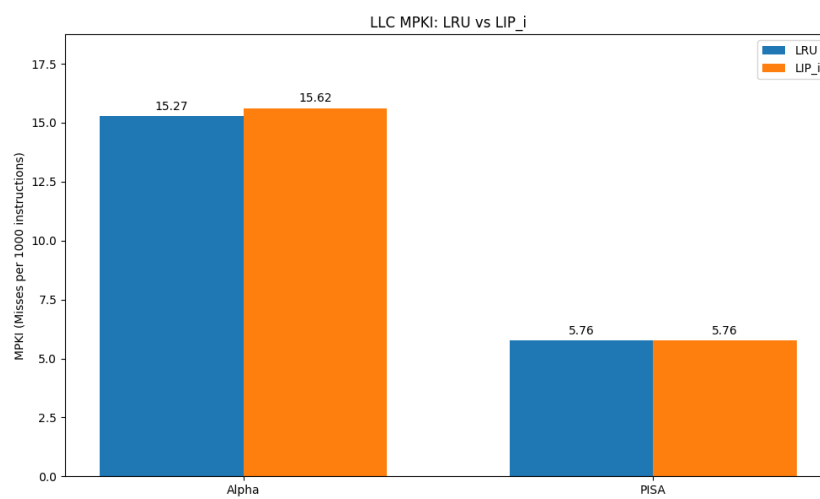


Fig.3b. LLC MPKI (Misses per Kilo Instructions) between LRU and LIP



## Conclusion

Under the tested experimental conditions:

- **LIP<sub>i</sub> (positions 1 and 11)** shows **no measurable performance advantage** over the traditional **LRU** replacement policy.
- The **identical performance statistics** are primarily attributed to the **small working set size** and **high temporal locality** of the *test-math* benchmark, which prevent meaningful cache contention or replacement activity.

## Work Contribution

Punnam Maakhish Sai

Worked on running the benchmark for Alpha ISA and implemented the LIP<sub>i</sub> cache replacement policy

Shivananda Reddy

Worked on running the benchmark for PISA ISA and gathered all the metric files compiled them and plotted the graphs.

The report is made by both of us together stating our appropriate results.

## Team Members

Punnam Maakhish Sai      B221135CS

Yapala Shivananda Reddy      B220603CS